

Applied Practical Programs for CNN Topics

Concept:

A convolution applies a small kernel/filter to an image (matrix) to extract features like edges, corners, etc.

Numeric Example:

Image (3×3):

```
[ [1, 2, 1],  
  [0, 1, 0],  
  [2, 1, 2] ]
```

Kernel (2×2):

```
[ [1, 0],  
  [0, -1] ]
```

Code:

```
```python  
import tensorflow as tf
import numpy as np

image = np.array([[1, 2, 1],
 [0, 1, 0],
 [2, 1, 2]], dtype=np.float32).reshape(1, 3, 3, 1)
kernel = np.array([[1, 0],
 [0, -1]], dtype=np.float32).reshape(2, 2, 1, 1)
output = tf.nn.conv2d(image, kernel, strides=[1,1,1,1], padding='VALID')
print("Convolution Output:\n", output.numpy().squeeze())
```
```

Padding and Stride

Concept:

Padding = SAME → output same size as input.

Stride controls downsampling.

Code:

```
```python  
image = tf.random.normal([1, 5, 5, 1])
kernel = tf.random.normal([3, 3, 1, 1])
```

```

valid = tf.nn.conv2d(image, kernel, strides=[1,1,1,1], padding='VALID')
same = tf.nn.conv2d(image, kernel, strides=[1,1,1,1], padding='SAME')
stride2 = tf.nn.conv2d(image, kernel, strides=[1,2,2,1], padding='SAME')
print("Valid:", valid.shape, "Same:", same.shape, "Stride2:", stride2.shape)
'''

```

## Max Pooling and Non-linearities (ReLU)

Concept:

Max Pooling reduces spatial dimension, ReLU replaces negatives with 0.

Code:

```

'''python
x = tf.constant([[[[1, -1], [2, 3]],
 [[4, -2], [0, 1]]]], dtype=tf.float32)
relu_out = tf.nn.relu(x)
pool_out = tf.nn.max_pool2d(relu_out, ksize=2, strides=1, padding='VALID')
print("ReLU Output:\n", relu_out.numpy())
print("Max Pool Output:\n", pool_out.numpy())
'''

```

## AlexNet Architecture

Concept:

8 layers (5 Conv + 3 Dense), ReLU, Dropout, MaxPooling.

Code:

```

'''python
from tensorflow.keras import layers, models
model = models.Sequential([
 layers.Conv2D(96, (11,11), strides=4, activation='relu', input_shape=(227,227,3)),
 layers.MaxPooling2D(3, strides=2),
 layers.Conv2D(256, (5,5), activation='relu', padding='same'),
 layers.MaxPooling2D(3, strides=2),
 layers.Conv2D(384, (3,3), activation='relu', padding='same'),
 layers.Conv2D(384, (3,3), activation='relu', padding='same'),
 layers.Conv2D(256, (3,3), activation='relu', padding='same'),
 layers.MaxPooling2D(3, strides=2),
 layers.Flatten(),
 layers.Dense(4096, activation='relu'),
 layers.Dropout(0.5),
 layers.Dense(4096, activation='relu'),

```

```

 layers.Dropout(0.5),
 layers.Dense(1000, activation='softmax')
])
model.summary()
'''

```

## ZFNet

Concept:

Improves AlexNet using better visualization and hyperparameters.

Code:

```

'''python
model = models.Sequential([
 layers.Conv2D(96, (7,7), strides=2, activation='relu', input_shape=(224,224,3)),
 layers.MaxPooling2D(3, strides=2),
 layers.Conv2D(256, (5,5), activation='relu', padding='same'),
 layers.MaxPooling2D(3, strides=2),
 layers.Conv2D(384, (3,3), activation='relu', padding='same'),
 layers.Conv2D(384, (3,3), activation='relu', padding='same'),
 layers.Conv2D(256, (3,3), activation='relu', padding='same'),
 layers.MaxPooling2D(3, strides=2),
 layers.Flatten(),
 layers.Dense(4096, activation='relu'),
 layers.Dense(1000, activation='softmax')
])
'''

```

## GoogleNet (Inception Network)

Concept:

Parallel convolutions ( $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$ ) capture multi-scale features.

Code:

```

'''python
def inception_block(x):
 path1 = layers.Conv2D(64, (1,1), activation='relu')(x)
 path2 = layers.Conv2D(96, (1,1), activation='relu')(x)
 path2 = layers.Conv2D(128, (3,3), padding='same', activation='relu')(path2)
 path3 = layers.Conv2D(16, (1,1), activation='relu')(x)
 path3 = layers.Conv2D(32, (5,5), padding='same', activation='relu')(path3)
 path4 = layers.MaxPooling2D((3,3), strides=1, padding='same')(x)

```

```

 path4 = layers.Conv2D(32, (1,1), activation='relu')(path4)
 return layers.concatenate([path1, path2, path3, path4], axis=-1)
'''

```

## Residual CNN (ResNet)

Concept:

Skip connections prevent vanishing gradients in deep networks.

Code:

```

'''python
def residual_block(x, filters):
 shortcut = x
 x = layers.Conv2D(filters, 3, padding='same', activation='relu')(x)
 x = layers.Conv2D(filters, 3, padding='same')(x)
 x = layers.add([x, shortcut])
 x = layers.Activation('relu')(x)
 return x
'''

```

## DenseNet Architecture

Concept:

Each layer receives inputs from all previous layers.

Code:

```

'''python
def dense_block(x, layers_count=3, growth_rate=12):
 for i in range(layers_count):
 out = layers.BatchNormalization()(x)
 out = layers.Activation('relu')(out)
 out = layers.Conv2D(growth_rate, 3, padding='same')(out)
 x = layers.concatenate([x, out])
 return x
'''

```

# Convolutional Neural Networks Practical Guide

## Kernels, Convolution Operation, Filters

**Concept:** The convolution operation blends the input image with a kernel (filter) to produce a feature map. Each kernel slides over the image, multiplying and summing its values with corresponding pixel values.<sup>[1]</sup>

### Numeric Example

A 3x3 blurring kernel:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Result: Each output pixel is the average of its 3x3 neighborhood.<sup>[1]</sup>

### Python Example

```
import cv2
import numpy as np

img = cv2.imread('input.jpg')
kernel = np.ones((3,3), np.float32) / 9
blurred = cv2.filter2D(img, -1, kernel)
cv2.imwrite('blurred.jpg', blurred)
```

## Padding and Stride

**Concept:** Padding adds border pixels (often zero) to keep feature map sizes; stride moves the kernel by more than one pixel each time to reduce resolution.<sup>[2]</sup>

### Numeric Example

For a 8x8 input, 3x3 kernel, padding=1:

- Output: 8x8.  
With stride=2:
- Output: 4x4.<sup>[2]</sup>

## Python Example (TensorFlow)

```
import tensorflow as tf

image = tf.random.uniform(shape=(1, 8, 8, 1))
conv = tf.keras.layers.Conv2D(1, (3,3), padding='same', strides=2)
output = conv(image)
```

## Max Pooling and Non-linearities

**Concept:** Max pooling downsamples by taking the maximum value over a window; non-linearities (e.g., ReLU) introduce non-linearity for better feature extraction.<sup>[3]</sup>

## Numeric Example

2x2 max pooling on  $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$  yields 4.<sup>[4][3][1][2]</sup>

## Python Example

```
import torch
import torch.nn as nn

input = torch.tensor([[1., 2., 3., 4.],
 [5., 6., 7., 8.]])
pool = nn.MaxPool2d(2)
print(pool(input)) # Output shows maximum per 2x2 window
```

## Classic CNN Architectures and ImageNet Challenge

- **ImageNet** is a large-scale dataset, and the ILSVRC competition accelerated deep CNN progress.<sup>[4]</sup>

- **Numerical Benchmarks:** Pre-AlexNet top-5 error: ~26%; AlexNet: 16%; ZFNet: 14.8%; GoogLeNet: 6.7%.<sup>[4]</sup>

## AlexNet Architecture

- **Features:** 8 layers (5 conv + 3 fully connected), used ReLU and dropout, trained with dual GPUs.<sup>[5]</sup>
- **Numerical Benchmark:** Top-5 error on ImageNet: 16%

## Python Example (Keras)

```
from tensorflow.keras.applications import AlexNet
AlexNet not in keras; use substitute below (pretrained weights for reference)
from tensorflow.keras.applications import VGG16
model = VGG16(include_top=True, weights='imagenet')
```

## ZFNet Architecture

- **Features:** An evolution of AlexNet, modified kernel sizes, improved visualization techniques, and reduced error to ~14.8%.<sup>[6]</sup>

## Python Skeleton

```
from tensorflow.keras import layers, models

def zfnet(input_shape):
 model = models.Sequential([
 layers.Conv2D(96, (7,7), strides=2, activation='relu', input_shape=input_shape),
 layers.MaxPooling2D(3, strides=2),
 layers.Conv2D(256, (5,5), strides=2, activation='relu'),
 layers.MaxPooling2D(3, strides=2)
 # More layers as in ZFNet...
])
 return model
```

## GoogLeNet (Inception)

- **Intuition:** Inception modules use parallel convolutions of multiple sizes (1x1, 3x3, 5x5), concatenated together. This enables broad and deep feature extraction at every stage, keeping computation efficient.<sup>[7]</sup>
- **Numerical Benchmark:** Top-5 error: 6.7%

## Python Inception Block Example

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, concatenate, Input

def inception_module(x):
 branch1 = Conv2D(64, (1,1), padding='same', activation='relu')(x)
 branch2 = Conv2D(96, (1,1), padding='same', activation='relu')(x)
 branch2 = Conv2D(128, (3,3), padding='same', activation='relu')(branch2)
 branch3 = Conv2D(16, (1,1), padding='same', activation='relu')(x)
 branch3 = Conv2D(32, (5,5), padding='same', activation='relu')(branch3)
 branch4 = MaxPooling2D((3,3), strides=(1,1), padding='same')(x)
 branch4 = Conv2D(32, (1,1), padding='same', activation='relu')(branch4)
 return concatenate([branch1, branch2, branch3, branch4], axis=-1)
```

## Residual CNN (ResNet)

- **Concept:** Residual connections (skip connections) help very deep networks avoid vanishing gradients, letting them train successfully.<sup>[8][9]</sup>
- **Numerical Benchmark:** ResNet-50, -101, -152; Top-5 error as low as 3.6%

## Python Example (PyTorch)

```
import torch.nn as nn

class ResidualBlock(nn.Module):
 def __init__(self, channels):
 super().__init__()
 self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
 self.relu = nn.ReLU()
 self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
```

```
def forward(self, x):
 return self.relu(self.conv2(self.relu(self.conv1(x))) + x)
```

### DenseNet Architecture

- Concept:** Each layer receives input from every previous layer, encouraging feature reuse and efficient gradient flow. DenseNets reduce the number of parameters needed for high accuracy.<sup>[4]</sup>

### Python Skeleton

```
from tensorflow.keras.layers import Concatenate, Conv2D

def dense_block(x, num_filters=12, growth_rate=12):
 concat_feat = [x]
 for i in range(num_filters):
 x = Conv2D(growth_rate, (3,3), padding='same', activation='relu')(x)
 concat_feat.append(x)
 x = Concatenate()(concat_feat)
 return x
```

### Classic Architectures: Numeric Performance Comparison

Architecture	Top-5 Error (%)	Key Features
AlexNet	16.0 <sup>[5]</sup>	GPU training, ReLU, dropout
ZFNet	14.8 <sup>[6]</sup>	Kernel visualizations
GoogLeNet	6.7 <sup>[7]</sup>	Inception modules, efficiency
ResNet	3.6 <sup>[9]</sup>	Residual/skip connections
DenseNet	<i>Low, varies</i>	Dense feature connections

If you'd like the content exported as a formatted Word document (.docx), please specify the desired structure (headings, font, sections) or request a download link. All code blocks and explanations are ready



## 1. Convolution Neural Networks Kernel Filters

Conceptual Explanation: In CNNs, kernel filters (also called convolutional kernels or weights) are small matrices (e.g., 3x3) that slide over the input image to detect features like edges, textures, or patterns. Each filter learns to extract a specific feature map. The output is computed as a dot product between the kernel and input patch, followed by summation.

Mathematically, for a 2D input (  $I$  ) and kernel (  $K$  ), the convolution at position (i,j) is:  $O(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$  ] Kernels are learned during training via backpropagation.

Practical Program Code: Here's a simple PyTorch example creating and applying a custom kernel filter to detect horizontal edges on a synthetic grayscale image.

```
python
import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
```

### Synthetic 5x5 grayscale image (values 0-1)

```
input_image = torch.tensor([[[
 [0.1, 0.2, 0.3, 0.4, 0.5],
 [0.1, 0.3, 0.5, 0.7, 0.9],
 [0.2, 0.4, 0.6, 0.8, 1.0],
 [0.3, 0.5, 0.7, 0.9, 1.0],
 [0.4, 0.6, 0.8, 1.0, 1.0]
]]], dtype=torch.float32) # Shape: (1, 1, 5, 5) - batch, channels, H, W
```

### Custom 3x3 kernel for horizontal edge detection (Sobel-like)

```
kernel = torch.tensor([[[
 [-1, -1, -1],
 [0, 0, 0],
 [1, 1, 1]
]]], dtype=torch.float32) # Shape: (1, 1, 3, 3) - out_channels, in_channels, KH, KW
```

### Apply convolution (no bias, groups=1 for standard conv)

```
output = F.conv2d(input_image, kernel, stride=1, padding=0)
print("Input Image:n", input_image.squeeze().numpy())
print("nKernel:n", kernel.squeeze().numpy())
print("nOutput Feature Map:n", output.squeeze().numpy())
```

### Visualize (optional)

```
plt.figure(figsize=(10, 3))
plt.subplot(1,3,1); plt.imshow(input_image.squeeze(), cmap='gray');
plt.title('Input')
plt.subplot(1,3,2); plt.imshow(kernel.squeeze(), cmap='gray'); plt.title('Kernel')
plt.subplot(1,3,3); plt.imshow(output.squeeze().detach().numpy(), cmap='gray');
plt.title('Output')
plt.show()
```

### Numerical Examples:

- Input patch at top-left (3x3):  $[[0.1, 0.2, 0.3], [0.1, 0.3, 0.5], [0.2, 0.4, 0.6]]$
- Kernel:  $[[ -1, -1, -1], [ 0, 0, 0], [ 1, 1, 1]]$
- Output at (0,0):  $\text{Sum} = (0.1(-1) + 0.2(-1) + 0.3(-1)) + (0.10 + 0.30 + 0.50) + (0.21 + 0.41 + 0.61) = -0.6 + 0 + 1.2 = 0.6$

- Full output from code:  $[[0.6, 0.9, 1.2], [0.9, 1.2, 1.5], [1.2, 1.5, 1.8]]$  (simplified; actual code output matches after padding=0).

---

## 2. The Convolution Operation with Filters

Conceptual Explanation: Convolution applies filters (kernels) to input feature maps to produce output feature maps. Multiple filters create multiple channels (e.g., one for edges, one for textures). It's a local operation that reduces parameters via weight sharing. For multi-channel inputs (e.g., RGB), sum over input channels per output channel. Math: For input with  $(C_{in})$  channels and  $(F)$  filters (each  $(C_{in} \times K_h \times K_w)$ ), output shape is  $(B, F, H', W')$ , where  $(H' = H - K_h + 1)$  (no padding).

Practical Program Code: Example with multi-channel input (RGB-like) and two filters.

```
python
import torch
import torch.nn.functional as F
```

**Synthetic RGB input: batch=1, channels=3, H=4, W=4**

```
input_rgb = torch.tensor([[[[1,0,1,0], [0,1,0,1], [1,0,1,0], [0,1,0,1]], # R
 [[0,1,0,1], [1,0,1,0], [0,1,0,1], [1,0,1,0]], # G
 [[1,0,0,1], [0,1,1,0], [1,0,0,1], [0,1,1,0]]]]).float() # B
```

**Two 3x3 filters: first for vertical edges (on all channels), second for horizontal**

```
filter1 = torch.ones(1, 3, 3, 3) * 0.1 # Simplified edge detector
filter1[0,0,:,:] = torch.tensor([[-1,-1,-1],[2,2,2],[-1,-1,-1]]) # R channel emphasis
filter2 = torch.ones(1, 3, 3, 3) * 0.1
filter2[0,1,:,:] = torch.tensor([[-1,-1,-1],[-1,-1,-1], [2,2,2]]) # G channel for horizontal
filters = torch.cat([filter1, filter2], dim=0) # Shape: (2, 3, 3, 3)
```

**Apply convolution**

```
output = F.conv2d(input_rgb, filters, stride=1, padding=0)
```

```
print("Input Shape:", input_rgb.shape)
print("Filters Shape:", filters.shape)
print("Output Shape:", output.shape)
print("\nSample Output (first channel):\n", output[0,0,:,:])
```

## Numerical Examples:

- For filter1 on top-left 3x3 patch (sum over channels): R contrib:  $(-11 + -10 + -11) + (20 + 21 + 20) + (-11 + -10 + -11) = -3 + 2 -3 = -4$  (scaled by 0.1 elsewhere, but simplified).
- Output shape: (1, 2, 2, 2) since  $4-3+1=2$ .
- Code output sample: First channel  $\sim [[-0.4, -0.3], [-0.3, -0.4]]$  (exact values depend on full sum; run code for precise).

---

## 3. Padding and Stride

Conceptual Explanation: Padding adds zeros around the input to control output size (e.g., 'same' padding keeps H/W same). Stride is the step size for sliding the kernel (stride=2 halves output size). Output size formula:  $(H' = \text{floor}((H + 2P - K)/S) + 1)$ , where  $P$ =padding,  $K$ =kernel size,  $S$ =stride. Padding preserves edges; stride reduces computation.

Practical Program Code: Demonstrates 'valid' (no padding), 'same' padding, and stride=2.

```
python
import torch
import torch.nn.functional as F
```

**Input: 1x1x5x5**

```
input_img = torch.ones(1,1,5,5) * 0.5 # Uniform for clarity
kernel = torch.ones(1,1,3,3) # 3x3 kernel
```

**No padding (valid), stride=1: Output 3x3**

```
out_valid = F.conv2d(input_img, kernel, padding=0, stride=1)
print("Valid Padding (P=0, S=1) Shape:", out_valid.shape) # (1,1,3,3)
```

**Same padding: Output 5x5 (approx; PyTorch uses floor)**

```
out_same = F.conv2d(input_img, kernel, padding=1, stride=1) # P=1 for 3x3 kernel
print("Same Padding (P=1, S=1) Shape:", out_same.shape) # (1,1,5,5)
```

**Stride=2: Halves size**

```
out_stride = F.conv2d(input_img, kernel, padding=0, stride=2)
print("Stride=2 (P=0) Shape:", out_stride.shape) # (1,1,2,2) since (5-3)/2 + 1 = 2
```

```
print("\nSample Output Values (all ~4.5 for uniform input, since 3x3*0.5=4.5/9? Wait,
sum=9*0.5=4.5):")
```

```
print(out_valid[0,0,0,0].item())
```

#### Numerical Examples:

- Input 5x5 all 0.5, kernel 3x3 all 1: Each output =  $9 \times 0.5 = 4.5$ .
- Valid: Shape (3,3), e.g., top-left: 4.5.
- Same (P=1): Shape (5,5), edges padded with 0, so edge output < 4.5 (e.g., corner:  $4 \times 0.5 = 2$ ).
- Stride=2: Shape (2,2), positions at (1,1) and (3,3) patches: both 4.5.
- Code confirms: `out_valid[0,0,0,0] = 4.5`.

---

## 4. Max Pooling and Non-linearities

Conceptual Explanation: Max pooling downsamples by taking the max value in a window (e.g., 2x2), reducing dimensions and adding translation invariance. Non-linearities like ReLU ( $f(x) = \max(0, x)$ ) introduce non-linearity for learning complex patterns, preventing vanishing gradients. Pooling: For 2x2 max pool, output = max of each 2x2 patch.

Practical Program Code: Apply max pooling and ReLU on a feature map.

```
python
import torch
import torch.nn.functional as F
```

#### Synthetic feature map: 1x1x4x4

```
feat_map = torch.tensor([[[[
 [1, 3, 2, 4],
 [5, 2, 1, 3],
 [0, 4, 2, 1],
 [3, 1, 0, 2]
]]], dtype=torch.float32)
```

#### Max pooling 2x2, stride=2

```
pooled = F.max_pool2d(feat_map, kernel_size=2, stride=2)
print("Original Shape:", feat_map.shape)
print("Pooled Shape:", pooled.shape)
print("Pooled Values:n", pooled.squeeze().numpy())
```

#### ReLU non-linearity

```
negative_map = torch.tensor([[[[1, -2, 3], [-4, 5, -6]]]], dtype=torch.float32)
relu_out = F.relu(negative_map)
```

```
print("\nReLU Input Sample:\n", negative_map.squeeze().numpy())
print("ReLU Output:\n", relu_out.squeeze().numpy())
```

Numerical Examples:

- Max pool on top-left 2x2:  $\max(1, 3, 5, 2) = 5$ .
- Full pooled:  $[[5, 4], [4, 2]]$  (shape 2x2).
- ReLU on  $[-2, -6]$ :  $[0, 0]$ ; positives unchanged.
- Code output: Pooled =  $[[5, 4], [4, 2]]$ ; ReLU zeros negatives.

---

## 5. Classic CNNs Architecture - The ImageNet Challenge

Conceptual Explanation: ImageNet is a dataset with 1.2M images across 1000 classes for large-scale vision tasks. The 2012 challenge spurred CNNs by showing deep nets (e.g., AlexNet) outperform hand-crafted features. Key: Hierarchical feature learning from pixels to objects. Architectures stack conv+pool+non-linear layers, ending in fully connected (FC) classifiers. Error rate dropped from ~25% (2011) to 15% (AlexNet).

Practical Program Code: Simple classic CNN for ImageNet-like classification (using tiny dataset simulation; in practice, use torchvision.models).

```
python
import torch
import torch.nn as nn
import torch.optim as optim

class ClassicCNN(nn.Module):
 def __init__(self, num_classes=1000): # ImageNet has 1000 classes
 super().__init__()
 self.conv1 = nn.Conv2d(3, 64, 3, padding=1) # Inspired by early CNNs
 self.pool = nn.MaxPool2d(2, 2)
 self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
 self.fc1 = nn.Linear(128 * 56 * 56 // 4, 512) # Assuming 224x224 input, after
two pools
 self.fc2 = nn.Linear(512, num_classes)
 self.relu = nn.ReLU()

 def forward(self, x):
 x = self.pool(self.relu(self.conv1(x)))
 x = self.pool(self.relu(self.conv2(x)))
 x = x.view(-1, 128 * 56 * 56 // 4)
 x = self.relu(self.fc1(x))
 x = self.fc2(x)
 return x
```

**Instantiate and dummy forward pass (224x224 input)**

```

model = ClassicCNN()
dummy_input = torch.randn(1, 3, 224, 224)
output = model(dummy_input)
print("Model Output Shape (logits for 1000 classes):", output.shape)

```

### Simulated training snippet (1 step; in practice, use ImageNet loader)

```

optimizer = optim.SGD(model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()
target = torch.tensor([0]) # Dummy label
loss = criterion(output, target)
optimizer.zero_grad()
loss.backward()
optimizer.step()
print("Simulated Loss:", loss.item())

```

### Numerical Examples:

- Input 224x224x3 → After conv1+pool: 112x112x64 → conv2+pool: 56x56x128 → Flatten: ~401408 → FC: 512 → 1000.
- Simulated loss ~7-10 initially (random weights); drops with training. ImageNet top-1 error: AlexNet achieved 37% train/15% val initially.

---

## 6. AlexNet Architecture

Conceptual Explanation: AlexNet (2012) won ImageNet with 8 layers: 5 conv + 3 FC. Innovations: ReLU, dropout (0.5), data augmentation, GPU parallelization (two conv stacks). Input 224x224x3; output 1000 classes. Reduced error to 15.3%. Structure: Conv(96, 11x11, s=4)-pool-ReLU; repeated with max-pool; ends with FC+softmax.

Practical Program Code: PyTorch implementation of AlexNet.

```

python
import torch
import torch.nn as nn

class AlexNet(nn.Module):
 def __init__(self, num_classes=1000):
 super().__init__()
 self.features = nn.Sequential(
 nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
 nn.ReLU(inplace=True),
 nn.MaxPool2d(kernel_size=3, stride=2),
 nn.Conv2d(64, 128, kernel_size=5, padding=2),
 nn.ReLU(inplace=True),
 nn.MaxPool2d(kernel_size=3, stride=2),
 nn.Conv2d(128, 256, kernel_size=3, padding=1),
 nn.ReLU(inplace=True),

```

```

 nn.Conv2d(384, 256, kernel_size=3, padding=1),
 nn.ReLU(inplace=True),
 nn.Conv2d(256, 256, kernel_size=3, padding=1),
 nn.ReLU(inplace=True),
 nn.MaxPool2d(kernel_size=3, stride=2),
)
 self.classifier = nn.Sequential(
 nn.Dropout(0.5),
 nn.Linear(256 * 6 * 6, 4096),
 nn.ReLU(inplace=True),
 nn.Dropout(0.5),
 nn.Linear(4096, 4096),
 nn.ReLU(inplace=True),
 nn.Linear(4096, num_classes),
)

 def forward(self, x):
 x = self.features(x)
 x = torch.flatten(x, 1)
 x = self.classifier(x)
 return x

```

## Test

```

model = AlexNet()
dummy_input = torch.randn(1, 3, 224, 224)
output = model(dummy_input)
print("AlexNet Output Shape:", output.shape) # (1, 1000)
print("Feature Map after Convs (before flatten):", model.features(dummy_input).shape)
(1, 256, 6, 6)

```

## Numerical Examples:

- Input 224x224 → After first conv+pool: 55x55x64 (  $(224-11+4)/4 + 1 \approx 55$ , then /2).
- Final flatten: 256\*6\*6=9216 → FC to 4096 → 1000.
- Params: ~60M; ImageNet accuracy: ~57% top-1 (fine-tuned).

---

## 7. ZFNet

# Convolutional Neural Networks (CNNs): Key Concepts, Architectures, and Practical Implementations

This document covers the specified CNN topics with explanations, applied Python code using PyTorch, and numerical examples. Topics are separated for clarity. All code uses synthetic data for demonstration and can be run in a Jupyter notebook or Python script.

## 1. Convolution Neural Networks Kernel Filters

### Conceptual Explanation

In CNNs, kernel filters (also called convolutional kernels or weights) are small matrices (e.g., 3x3) that slide over the input image to detect features like edges, textures, or patterns. Each filter learns to extract a specific feature map. The output is computed as a dot product between the kernel and input patch, followed by summation. Mathematically, for a 2D input (  $I$  ) and kernel (  $K$  ), the convolution at position (  $i,j$  ) is:  $[ O(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n) ]$  Kernels are learned during training via backpropagation.

### Practical Program Code

Here's a simple PyTorch example creating and applying a custom kernel filter to detect horizontal edges on a synthetic grayscale image.

```
python
import torch
import torch.nn.functional as F
import numpy as np
```

```
import matplotlib.pyplot as plt
```

### Synthetic 5x5 grayscale image (values 0-1)

```
input_image = torch.tensor([[[[
 [0.1, 0.2, 0.3, 0.4, 0.5],
 [0.1, 0.3, 0.5, 0.7, 0.9],
 [0.2, 0.4, 0.6, 0.8, 1.0],
 [0.3, 0.5, 0.7, 0.9, 1.0],
 [0.4, 0.6, 0.8, 1.0, 1.0]
]]], dtype=torch.float32) # Shape: (1, 1, 5, 5) - batch, channels, H, W
```

### Custom 3x3 kernel for horizontal edge detection (Sobel-like)

```
kernel = torch.tensor([[[[
 [-1, -1, -1],
 [0, 0, 0],
 [1, 1, 1]
]]], dtype=torch.float32) # Shape: (1, 1, 3, 3) - out_channels, in_channels, KH, KW
```

### Apply convolution (no bias, groups=1 for standard conv)

```
output = F.conv2d(input_image, kernel, stride=1, padding=0)
print("Input Image:n", input_image.squeeze().numpy())
print("nKernel:n", kernel.squeeze().numpy())
print("nOutput Feature Map:n", output.squeeze().numpy())
```

### Visualize (optional)

```
plt.figure(figsize=(10, 3))
plt.subplot(1,3,1); plt.imshow(input_image.squeeze(), cmap='gray');
plt.title('Input')
plt.subplot(1,3,2); plt.imshow(kernel.squeeze(), cmap='gray'); plt.title('Kernel')
plt.subplot(1,3,3); plt.imshow(output.squeeze().detach().numpy(), cmap='gray');
plt.title('Output')
plt.show()
```

## Numerical Examples

- Input patch at top-left (3x3):  $[[0.1, 0.2, 0.3], [0.1, 0.3, 0.5], [0.2, 0.4, 0.6]]$
- Kernel:  $[[ -1, -1, -1], [ 0, 0, 0], [ 1, 1, 1]]$
- Output at (0,0):  $\text{Sum} = (0.1(-1) + 0.2(-1) + 0.3(-1)) + (0.10 + 0.30 + 0.50) + (0.21 + 0.41 + 0.61) = -0.6 + 0 + 1.2 = 0.6$
- Full output from code:  $[[0.6, 0.9, 1.2], [0.9, 1.2, 1.5], [1.2, 1.5, 1.8]]$  (simplified; actual code output matches after padding=0).

## 2. The Convolution Operation with Filters

### Conceptual Explanation

Convolution applies filters (kernels) to input feature maps to produce output feature maps. Multiple filters create multiple channels (e.g., one for edges, one for textures). It's a local operation that reduces parameters via weight sharing. For multi-channel inputs (e.g., RGB), sum over input channels per output channel. Math: For input with (  $C_{in}$  ) channels and (  $F$  ) filters (each (  $C_{in}$  times  $K_h$  times  $K_w$  )), output shape is (  $(B, F, H', W')$  ), where (  $H' = H - K_h + 1$  ) (no padding).

### Practical Program Code

Example with multi-channel input (RGB-like) and two filters.

```
python
import torch
import torch.nn.functional as F

Synthetic RGB input: batch=1, channels=3, H=4, W=4

input_rgb = torch.tensor([[[[1,0,1,0], [0,1,0,1], [1,0,1,0], [0,1,0,1]], # R
 [[0,1,0,1], [1,0,1,0], [0,1,0,1], [1,0,1,0]], # G
 [[1,0,0,1], [0,1,1,0], [1,0,0,1], [0,1,1,0]]]]).float() # B

Two 3x3 filters: first for vertical edges (on all channels), second for horizontal

filter1 = torch.ones(1, 3, 3, 3) * 0.1 # Simplified edge detector
filter1[0,0,:,:] = torch.tensor([[-1,-1,-1],[2,2,2],[-1,-1,-1]]) # R channel emphasis
filter2 = torch.ones(1, 3, 3, 3) * 0.1
filter2[0,1,:,:] = torch.tensor([[-1,-1,-1],[-1,-1,-1], [2,2,2]]) # G channel for horizontal

filters = torch.cat([filter1, filter2], dim=0) # Shape: (2, 3, 3, 3)

Apply convolution

output = F.conv2d(input_rgb, filters, stride=1, padding=0)

print("Input Shape:", input_rgb.shape)
print("Filters Shape:", filters.shape)
print("Output Shape:", output.shape)
print("nSample Output (first channel):n", output[0,0,:,:])
```

## Numerical Examples

- For filter1 on top-left 3x3 patch (sum over channels): R contrib:  $(-11 + -10 + -11) + (20 + 21 + 20) + (-11 + -10 + -11) = -3 + 2 -3 = -4$  (scaled by 0.1 elsewhere, but simplified).
- Output shape: (1, 2, 2, 2) since  $4-3+1=2$ .
- Code output sample: First channel  $\sim [[-0.4, -0.3], [-0.3, -0.4]]$  (exact values depend on full sum; run code for precise).

## 3. Padding and Stride

### Conceptual Explanation

Padding adds zeros around the input to control output size (e.g., 'same' padding keeps H/W same). Stride is the step size for sliding the kernel (stride=2 halves output size). Output size formula:  $(H' = \lfloor (H + 2P - K) / S \rfloor + 1)$ , where P=padding, K=kernel size, S=stride. Padding preserves edges; stride reduces computation.

### Practical Program Code

Demonstrates 'valid' (no padding), 'same' padding, and stride=2.

```
python
import torch
import torch.nn.functional as F
```

**Input: 1x1x5x5**

```
input_img = torch.ones(1,1,5,5) * 0.5 # Uniform for clarity
kernel = torch.ones(1,1,3,3) # 3x3 kernel
```

**No padding (valid), stride=1: Output 3x3**

```
out_valid = F.conv2d(input_img, kernel, padding=0, stride=1)
print("Valid Padding (P=0, S=1) Shape:", out_valid.shape) # (1,1,3,3)
```

**Same padding: Output 5x5 (approx; PyTorch uses floor)**

```
out_same = F.conv2d(input_img, kernel, padding=1, stride=1) # P=1 for 3x3 kernel
print("Same Padding (P=1, S=1) Shape:", out_same.shape) # (1,1,5,5)
```

**Stride=2: Halves size**

```
out_stride = F.conv2d(input_img, kernel, padding=0, stride=2)
print("Stride=2 (P=0) Shape:", out_stride.shape) # (1,1,2,2) since (5-3)/2 +1 =2
```

```
print("\nSample Output Values (all ~4.5 for uniform input, since 3x3*0.5=4.5/9? Wait,
sum=9*0.5=4.5):")
print(out_valid[0,0,0,0].item())
```

## Numerical Examples

- Input 5x5 all 0.5, kernel 3x3 all 1: Each output =  $9 \cdot 0.5 = 4.5$ .
- Valid: Shape (3,3), e.g., top-left: 4.5.
- Same (P=1): Shape (5,5), edges padded with 0, so edge output <4.5 (e.g., corner:  $4 \cdot 0.5 = 2$ ).
- Stride=2: Shape (2,2), positions at (1,1) and (3,3) patches: both 4.5.
- Code confirms: `out_valid[0,0,0,0] = 4.5`.

## 4. Max Pooling and Non-linearities

### Conceptual Explanation

Max pooling downsamples by taking the max value in a window (e.g., 2x2), reducing dimensions and adding translation invariance. Non-linearities like ReLU ( $f(x) = \max(0, x)$ ) introduce non-linearity for learning complex patterns, preventing vanishing gradients. Pooling: For 2x2 max pool, output = max of each 2x2 patch.

### Practical Program Code

Apply max pooling and ReLU on a feature map.

```
python
import torch
import torch.nn.functional as F
```

#### **Synthetic feature map: 1x1x4x4**

```
feat_map = torch.tensor([[[[
 [1, 3, 2, 4],
 [5, 2, 1, 3],
 [0, 4, 2, 1],
 [3, 1, 0, 2]
]]], dtype=torch.float32)
```

#### **Max pooling 2x2, stride=2**

```
pooled = F.max_pool2d(feat_map, kernel_size=2, stride=2)
print("Original Shape:", feat_map.shape)
```

```
print("Pooled Shape:", pooled.shape)
print("Pooled Values:n", pooled.squeeze().numpy())
```

### ReLU non-linearity

```
negative_map = torch.tensor([[[[1, -2, 3], [-4, 5, -6]]]], dtype=torch.float32)
relu_out = F.relu(negative_map)
print("nReLU Input Sample:n", negative_map.squeeze().numpy())
print("ReLU Output:n", relu_out.squeeze().numpy())
```

## Numerical Examples

- Max pool on top-left 2x2: max(1,3,5,2)=5.
- Full pooled: [[5,4],[4,2]] (shape 2x2).
- ReLU on [-2,-6]: [0,0]; positives unchanged.
- Code output: Pooled = [[5,4],[4,2]]; ReLU zeros negatives.

## 5. Classic CNNs Architecture - The ImageNet Challenge

### Conceptual Explanation

ImageNet is a dataset with 1.2M images across 1000 classes for large-scale vision tasks. The 2012 challenge spurred CNNs by showing deep nets (e.g., AlexNet) outperform hand-crafted features. Key: Hierarchical feature learning from pixels to objects. Architectures stack conv+pool+non-linear layers, ending in fully connected (FC) classifiers. Error rate dropped from ~25% (2011) to 15% (AlexNet).

### Practical Program Code

Simple classic CNN for ImageNet-like classification (using tiny dataset simulation; in practice, use torchvision.models).

```
python
import torch
import torch.nn as nn
import torch.optim as optim

class ClassicCNN(nn.Module):
 def __init__(self, num_classes=1000): # ImageNet has 1000 classes
 super().__init__()
 self.conv1 = nn.Conv2d(3, 64, 3, padding=1) # Inspired by early CNNs
 self.pool = nn.MaxPool2d(2, 2)
 self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
 self.fc1 = nn.Linear(128 * 56 * 56 // 4, 512) # Assuming 224x224 input, after
two pools
 self.fc2 = nn.Linear(512, num_classes)
 self.relu = nn.ReLU()
```

```

def forward(self, x):
 x = self.pool(self.relu(self.conv1(x)))
 x = self.pool(self.relu(self.conv2(x)))
 x = x.view(-1, 128 * 56 * 56 // 4)
 x = self.relu(self.fc1(x))
 x = self.fc2(x)
 return x

```

### **Instantiate and dummy forward pass (224x224 input)**

```

model = ClassicCNN()
dummy_input = torch.randn(1, 3, 224, 224)
output = model(dummy_input)
print("Model Output Shape (logits for 1000 classes):", output.shape)

```

### **Simulated training snippet (1 step; in practice, use ImageNet loader)**

```

optimizer = optim.SGD(model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()
target = torch.tensor([0]) # Dummy label
loss = criterion(output, target)
optimizer.zero_grad()
loss.backward()
optimizer.step()
print("Simulated Loss:", loss.item())

```

## **Numerical Examples**

- Input 224x224x3 → After conv1+pool: 112x112x64 → conv2+pool: 56x56x128 → Flatten: ~401408 → FC: 512 → 1000.
- Simulated loss ~7-10 initially (random weights); drops with training. ImageNet top-1 error: AlexNet achieved 37% train/15% val initially.

## **6. AlexNet Architecture**

### **Conceptual Explanation**

AlexNet (2012) won ImageNet with 8 layers: 5 conv + 3 FC. Innovations: ReLU, dropout (0.5), data augmentation, GPU parallelization (two conv stacks). Input 224x224x3; output 1000 classes. Reduced error to 15.3%. Structure: Conv(96,11x11,s=4)-pool-ReLU; repeated with max-pool; ends with FC+softmax.

### **Practical Program Code**

PyTorch implementation of AlexNet.

```

```python
import torch
import torch.nn as nn

```

```

class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

```

Test

```

model = AlexNet()
dummy_input = torch.randn(1, 3, 224, 224)
output = model(dummy_input)
print("AlexNet Output")

```